

Big-O notation: time complexity

OPTIMIZING CODE IN JAVA



Pavlos Kosmetatos

Lead Engineer @Wealthyhood

What is time complexity?

- Time Complexity: Measure of how runtime grows with input size
- Helps answer: "What happens when my data gets 10× larger?"
- Does not focus on absolute time

Big-O notation

Mathematical notation to describe worst-case scenario.

Some common complexity classes:

- $O(1)$: Constant time - size-independent

Understanding $O(1)$

- An example of a $O(1)$ operation is `ArrayList`'s `get()`

Internal (simplified) implementation for an `ArrayList` that holds `Strings` :

```
public class ArrayList {}  
    private String[] data; // Internal array  
    private int size;  
  
    // Get operation - direct array access  
    public String get(int index) {  
        return data[index]; //  $O(1)$   
    }  
}
```

Big-O notation

Mathematical notation to describe worst-case scenario

Some common complexity classes:

- $O(1)$: Constant time - size-independent
- $O(n)$: Linear time - grows with input size

Understanding $O(n)$

- A similar example, but for `ArrayList`'s `contains()`

```
public boolean contains(Object o) {  
    return indexOf(o) >= 0;  
}  
  
public int indexOf(Object o) {  
    // Linear search through array  
    for (int i = 0; i < size; i++) {  
        if (o.equals(elementData[i])) {  
            return i;  
        }  
    }  
    return -1; // Not found  
}
```

Big-O notation

Mathematical notation to describe worst-case scenario

Some common complexity classes:

- $O(1)$: Constant time - size-independent
- $O(n)$: Linear time - grows with input size
- $O(n^2)$: Quadratic time - grows quadratically with input size

A practical example with quadratic complexity

```
// Finding a pair of numbers that sum to a target value  
// Time complexity:  $O(n^2)$ 
```

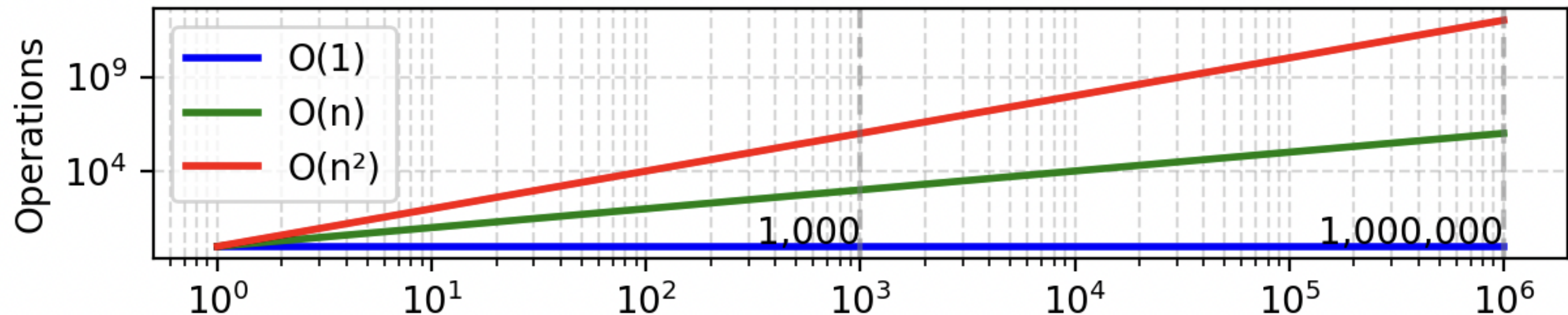
```
public int[] findPairWithSum(ArrayList<Integer> numbers, int targetSum) {  
    for (int i = 0; i < numbers.size(); i++) {  
        for (int j = i + 1; j < numbers.size(); j++) {  
            if (numbers.get(i) + numbers.get(j) == targetSum) {  
                console.log("Found them!")  
            }  
        }  
    }  
}
```


Why time complexity matters

Input size impact:

- $O(1)$: 1,000 -> 1,000,000 items = Same time!
- $O(n)$: 1,000 -> 1,000,000 items = 1,000× slower
- $O(n^2)$: 1,000 -> 1,000,000 items = 1,000,000× slower

Impact of Input Size on Different Time Complexities



Let's practice!

OPTIMIZING CODE IN JAVA

Big-O notation: space complexity

OPTIMIZING CODE IN JAVA



Pavlos Kosmetatos

Lead Engineer @Wealthyhood

What is space complexity?

- Time complexity describes how input size affects **runtime**
- Space complexity describes how input size affect **memory usage**

Understanding space complexity is crucial for building applications that:

- Only use the right amount of memory, and not more
- Avoid crashing with errors such as `OutOfMemoryError`

Big-O Notation

Notation is identical to time complexity.

Some common complexity classes:

- $O(1)$: Constant time - size-independent
- $O(n)$: Linear time - grows with input size
- $O(n^2)$: Quadratic time - grows quadratically with input size

A maximum-finder method

```
public int findMax(int[] array) {  
    int max = Integer.MIN_VALUE;  
    for (int value : array) {  
        if (value > max) {  
            max = value;  
        }  
    }  
    return max;  
}
```

- Whether our array of integers has 10 elements or 10 million, we still only use memory for one single variable, `max`
- The space complexity is $O(1)$ or constant space

A doubling method

```
public int[] doubleValues(int[] array) {  
    int[] result = new int[array.length];  
    for (int i = 0; i < array.length; i++) {  
        result[i] = array[i] * 2;  
    }  
    return result;  
}
```

- If our input has n elements, we need space for n additional elements
- The space complexity is $O(n)$ because the extra memory needed grows linearly with the input size

A multiplication table method

```
public int[][] multiplicationTable(int n) {  
    int[][] table = new int[n][n];  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < n; j++) {  
            table[i][j] = (i + 1) * (j + 1);  
        }  
    }  
    return table;  
}
```

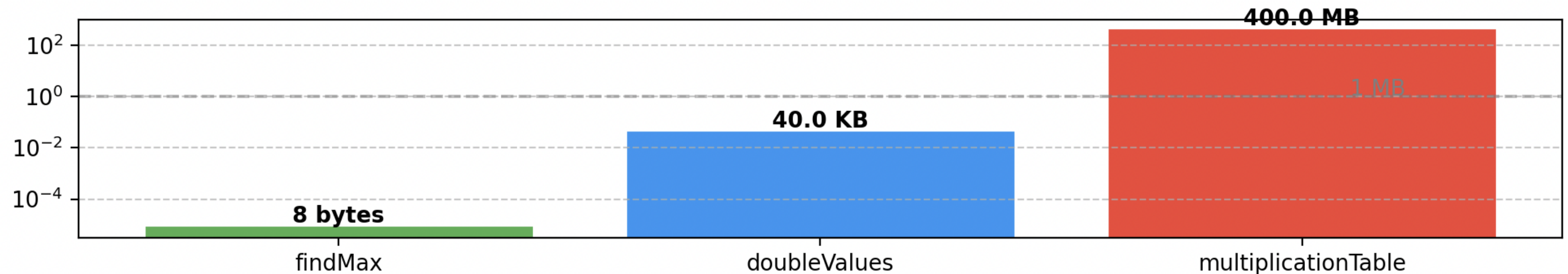
- If `n` is 10, we need 100 cells; if `n` is 100, we need 10,000 cells
- We classify this as $O(n^2)$

Why is space complexity important?

Memory is a finite resource!

Our previous examples in action, for an input size of 10,000 elements:

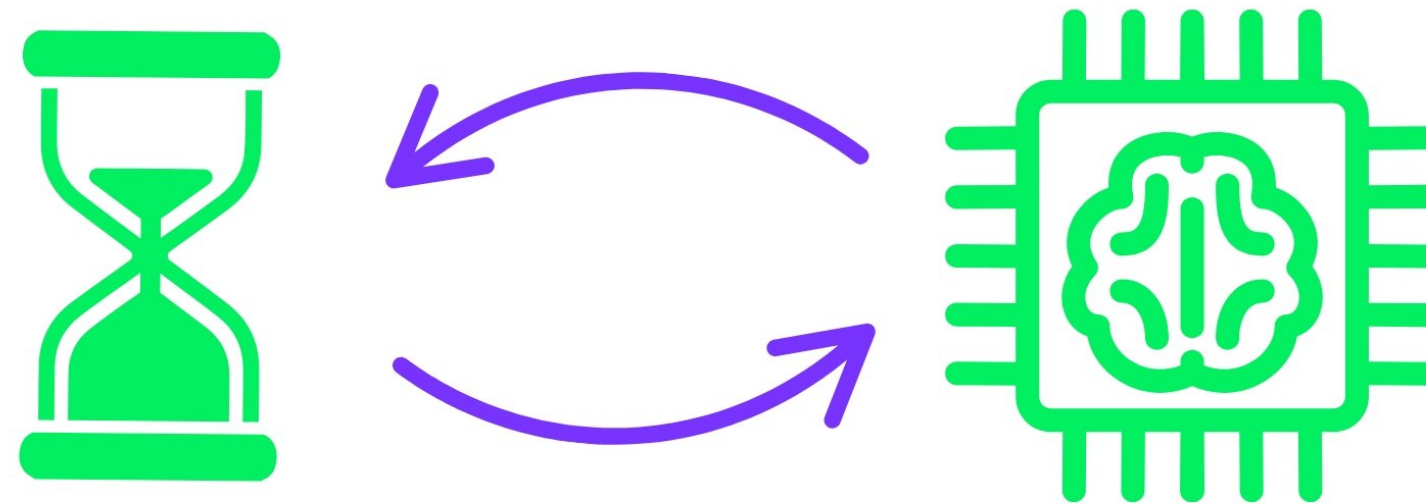
- `findMax` , $O(1)$ -> just a few extra bytes
- `doubleValues` , $O(n)$ -> around 40KB of extra memory
- `multiplicationTable` , $O(n^2)$ -> around **400MB** of extra memory



Space complexity vs. time complexity

Don't forget:

- Sometimes we trade space for time
- Sometimes we trade time for space
- The right choice depends on your specific constraints



Let's practice!

OPTIMIZING CODE IN JAVA

Efficient data structures: Sets & Maps

OPTIMIZING CODE IN JAVA



Pavlos Kosmetatos
Lead Engineer @Wealthyhood

Applying space/time complexity

We now know about space and time complexity!

How do we apply this understanding to write more efficient code?

By choosing the right data structure!

A user management system

- We are building a user management system
- We need to check, for a given username, if a user **exists**
- Using a list: complexity $O(n)$

```
public boolean usernameExists(ArrayList<String> users, String newUsername) {  
    for (String username : users) {  
        if (username.equals(newUsername)) {  
            return true;  
        }  
    }  
    return false;  
}
```

Sets

- A collection of unique elements with fast lookup times
- $O(1)$ average time complexity for adding, removing, and checking if an element exists

Improved user management system solution:

```
public class UserRegistry {  
    private HashSet<String> users = new HashSet<>();  
  
    public boolean userExists(String username) {  
        return users.contains(username); //  $O(1)$  average time  
    }  
}
```

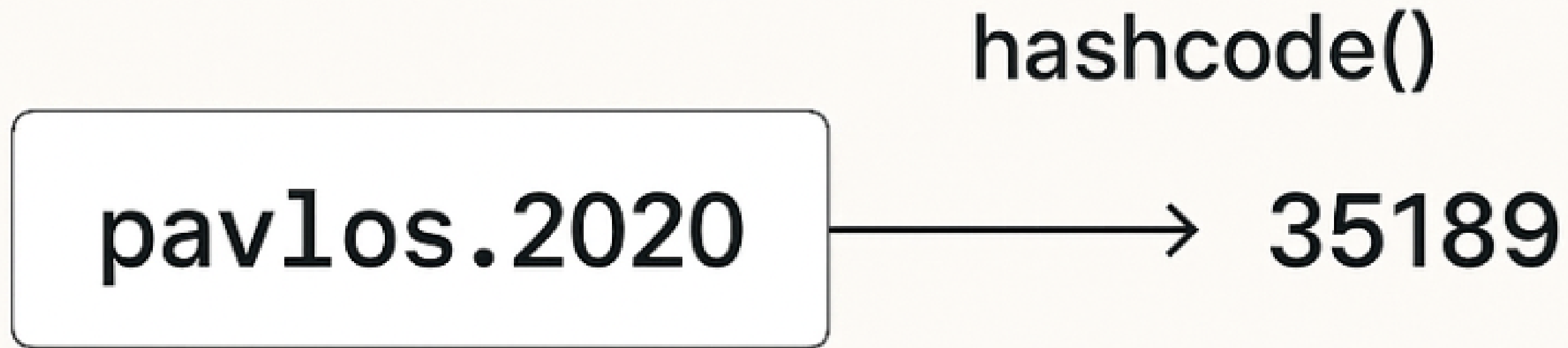

Maps

- Like a dictionary, word (key) -> definition (value)
- `HashMap` : $O(1)$ average time complexity for operations

```
public class UserCache {  
    private HashMap<String, UserProfile> userProfiles = new HashMap<>();  
  
    public UserProfile getUser(String username) {  
        return userProfiles.get(username); //  $O(1)$  average time  
    }  
}
```


HashCode method

- What makes those fast -> `hashCode()`
- In our `ArrayList` example, we did not know the index of the username to find
- Using the `hashCode()` method, we can convert an object into a index we can look for
- For example `pavlos.2020` -> `35189`



Indexing elements

- `HashMap` and `HashSet` have an underlying array
- When we add an element:
 - Java calls `hashCode()` on your element to get an integer
 - It calls modulo on the above to get the bucket number

Example:

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
```

```
"optimizingCodeInJava" -> 1406313774 // Implemented by Java
```

```
1406313774 % 16 = 14 <- that's our bucket!
```

Collisions

- When more than one items end up in the same bucket
 - Called a collision
- `LinkedList` for the bucket
- That's why we say $O(1)$ on average

A note to remember

Data structure selection is like choosing the right tool for a job - a hammer (`ArrayList`) is great for nails but terrible for screws (where a `Set` might work better).



Let's practice!

OPTIMIZING CODE IN JAVA